第一讲

开场引入

非常感谢大家愿意来听我们《这就是开源》的第一讲课程，因为这是我们本系列的第一节课程，所以这一节课主要是讲解一些关于计算机最基础的知识以及一些历史故事，这节课不会讲技术上知识，而是讲一些意识上的你应该对开源有的了解，所以对于大二大三的同学可能会有一些无聊，对于大一的有些同学可能也是这样，毕竟这是第一节课，不能上来就给大家上难度对吧，那样对高考之前没怎么接触过计算机的同学也不友好，比如说当时的我，我们还是希望这门课程几乎对所有人都是友好的，都是有帮助的。

OK，这门课程分为两大部分，前一部分是一些基础课，主要讲一些大家未来一定会用到的计算机工具，主要是参照了MIT的《你缺失的那门计算机课》The Missing Semester of Your CS Education，但是为了使这门课程更加适合我们的需要，我们这个课程的前三讲是没有参照这个课程的，前三讲是我们自己计划的。另外，关于这个课程，国内有重名版本，但是那个主要是针对于零门槛的非计算机专业学生，比如说怎么使用office，对于作为计算机专业的学生的我们显然是不够的。然后我们这个课程的后半部分会讲一些比较高级的技能，届时科协会请一些大佬来讲。如果有的同学你觉得你的基础已经很好了，那么可以课程进行到后半部分的时候再来听。

然后在这一节课的最后呢，我会对问卷里的一些建议展开一些锐评，换句话说就是开炮，我已经忍了好几天了。

OK下面我们开始这门课。

计算机里面有什么

大家高考结束了很多人都会去选择买一台新电脑对吧，你在买电脑的时候导购都会跟你讲，这台电脑用了什么样的处理器，有多大的内存，用了多大的硬盘，屏幕分辨率是几点几k的等等等等，那实际上电脑最重要的三个部件就是处理器，内存和硬盘。这里我们使用厨房来做了一个比喻，处理器起到了一个非常核心的作用，他是最实际的执行者，埋头苦干，然后是内存，用来暂时存放正在运行的程序，所以内存的大小是很重要的，内存大一些能放的就多，内存小一些能放的就少，最后是硬盘，用来长期保存你的文件与程序，如果你经常玩一些大型游戏或者经常拍照拍视频，你可能就需要一个大一点的硬盘，反之硬盘就可以小一些。

下面这三张图片是我自己的电脑的情况，各位也可以打开自己电脑上的任务管理器，大家能看到我的电脑用到了13代处理器，40GDDR5内存以及一个2000G的硬盘，这个因人而异。

为什么我们需要操作系统

我们在拿到电脑开机的时候都会看到一个界面对吧，没有人新买的电脑就是黑屏的对吧，大家肯定知道我们刚开机看到的那个玩意叫做操作系统，有的同学使用windows，有的同学使用mac，还有的同学可能使用鸿蒙对吧，那我们看到的这个系统它到底有什么作用呢？

下面我们来考虑这样一个情形：你的导员在QQ上给你发了文件，你的部长在微信上给你发了通知，你的同学在抖音上给你转发了视频...那处理器该先给谁用呢？处理器就一个，总不能让QQ一直占用着处理器不撒手对吧，那这样的话微信和抖音就没有办法运行了，那内存该怎么分配呢？内存就那么点，16G32G的，也不能一股脑全给QQ对吧，那你让微信的数据往哪里放。所以我们需要一个管理者，它能够安排程序何时使用处理器，能够合理分配内存，还要尽可能避免不同程序之间的相互干扰。这个管理者，就是操作系统，我们刚才提到的这些呢，就是操作系统的第一个大方面。

但是操作系统实现的功能不止这些，就比如有人在微信上给你发送了一张图片，当你点击保存的时候，程序会将图片保存到硬盘上，但是程序该怎么操作硬盘呢？我们都知道市场上有各种各样的硬盘厂家，它们可能使用了不同的技术不同的工艺，那你要让微信这个程序来操作不同的硬件，那我估计微信的开发者要大骂特骂了，因为这样会极大的增加开发成本，所以我们不让微信来处理保存图片这件事，微信只需要请求“写入文件”，剩下的事情就由操作系统来完

成了，那操作系统的开发者会不会骂人呢？不会的，如果程序直接操作硬件，是可能会产生大麻烦的。这就是操作系统的第二个大方面：为应用程序提供统一的服务，让程序不必直接处理复杂的硬件细节。

什么是OS

综上，什么是操作系统呢？操作系统是管理计算机硬件与软件资源，并为应用程序提供运行环境和基础服务的系统软件。一方面，它是资源管理者，能够安排处理器工作，分配内存空间以及管理文件和设备，另一方面，它也是程序服务者，帮助程序读写文件，提供统一的设备接口并且为程序提供运行环境。操作系统既是资源的管理者，也是连接应用程序与计算机硬件之间的桥梁。

OK，接下来我们要开始讲历史小故事了，大家一定要珍惜哈(笑死我了)，只有第一节课有这种历史故事，后面的课程，都不会有历史故事了，我那天在写后面课程的大纲，发现一句废话都出现不了，而且一句废话不说都不一定能在规定时间内讲完，所以大家一定要珍惜今晚，嗯(真的笑死我了)。

操作系统发展史

这是世界上早期的电子计算机，非常庞大对吧，好像是给美军计算火炮的火力范围用的，这个时候的计算机还没有什么操作系统，唯一的任务就是计算，但是也正是因为没操作系统，程序的装载、准备和运行都需要大量的人工参与，十分复杂。

后来人们发现这样是不行的，计算机算的这么快，就应该让它多算，一直算下去，它就不应该休息，所以这个时候人们就想，能不能让计算机完成一个任务之后，自动地开始执行下一个任务呢？当然是可以的，这就是我们的单道批处理系统，之所以叫批处理，是因为它能够自动组织并依次执行一批作业，减少人工的干预，提高计算机的使用效率，之所以叫单道，是因为它还是一次只能运行一个作业，运行完一个才能运行下一个。

但是后来人们发现这样还不够，因为程序运行的过程中可能会停下来等待新的数据输入，但是等待数据输入的时候计算机肯定是没办法运行的，就停在那里，所以人们又设计了多道批处理系统，在作业A不得不暂停的时候运行作业B，如图所示，根本目的其实还是为了提高计算机的使用效率，谁让它算的快呢对吧。

再之后，随之电子计算机逐渐走进各种大学和研究机构，同一台电子计算机会被多个人共同使用，因为当时的计算机体积很大而且售价及其高昂，根本不可能一人一台，这个时候批处理系统其实就有一些不合时宜了，因为批处理系统运行任务完全取决于当前的任务队列，那用户提交了作业之后就必须得等着，这就有些麻烦了，如果我写程序的时候一不小心写错了，运行运行着发现报错了，给我退回来之后我修改完就要重新提交，重新等待，那得等到猴年马月对吧，所以这时出现了分时操作系统，它把处理器的时间切分成一个又一个时间片，让多个用户的程序轮流使用处理器，这样的话对每个用户都是公平的，每个用户的程序都能及时得到运行，并且获得较为及时的响应。

我们刚才说的这些操作系统其实大家都不用记，只需要知道历史上曾经有过这些东西就可以了，这些老古董某种程度上其实都过时了。OK我们继续。

在20 世纪 60 年代，MIT、贝尔实验室与通用电气公司合作开发了一套名为 Multics 的分时操作系统。但是，随着系统越来越复杂，开发难度和成本也越来越高，开发进度不及预期，最后贝尔实验室退出了该项目。但是，但是，尽管贝尔实验室退出了这个项目，这个系统最后还是做出来了，并且对现代操作系统有着不小的影响，为什么这个操作系统的开发难度与成本越来越高呢？首先是因为这个操作系统有着很多超前的设计，其次是因为这三个机构仗着自己力量强大，非得要一开始就很完美地实现它，所以就导致开发过程越来越难。

所以后来，贝尔实验室尽管退出了这个项目，但是里面有少数的研究人员依然对开发操作系统这件事念念不忘，所以它充分吸取了经验教训，不追求一开始就实现大量复杂功能，而是先构建一套小巧、实用、可以不断扩展的系统，也就是后来的UNIX。

这个UNIX依然支持多用户，也就是我们刚才提到过的分时操作系统，再加上它吸取了Multics的经验教训，所以UNIX的设计相当的简洁，后来，贝尔实验室的研究人员使用C语言把UNIX当中的部分组件给重写了，大一的同学已经学过C++了对吧，我们都知道这样重写就会使得其具有可移植性，对硬件的依赖程度就降低了，这三个特性极大地促进了UNIX的传播。

所以这个UNIX凭借优秀的设计与良好的可移植性，逐渐从研究机构走向大学与商业市场，吸引了越来越多的开发者和企业参与。那为什么早期的一些大学和研究机构能获得UNIX的源代码呢？这是因为贝尔实验室在当时还隶属于AT&T，也就是美国电话电报公司，这个美国电话电报公司在上个世界七八十年代可谓是声名显赫对吧，以至于美国政府和这个公司签署了反垄断协议，限制了这个公司在电信业务之外的商业活动，所以它没有办法通过贩卖UNIX来获取利润，所以它只好以非常非常低的价格向这些大学和研究机构出售代码的许可，这某种程度上也促进了UNIX系统的发展。

随着UNIX系统的快速发展，很多计算机厂家就嗅到了商机，他们花钱拿到UNIX代码的使用权修改权以及销售权，就开始在原来的基础上为自己的UNIX系统增加独有的功能和接口，但是这个时候就出现问题了，这些系统虽然都属于UNIX家族，但是却不一定能兼容彼此的软件，问题就出在厂商对自己的UNIX系统做出的改动上，他们就是有些独断专行吧，不管同行的死活，这样的话对于开发者和用户其实是很不友好的，对开发者而言，同一个程序可能需要针对不同的UNIX系统进行反复修改和适配，对于用户来说，更换系统可能意味着高昂的迁移成本，以及对特定的厂商的依赖，换句话说，没有哪个厂商不想垄断市场，所以搞了这些幺蛾子，但是最后羊毛出在了羊身上，全被开发者和用户买单了，因为兼容性的问题，各大厂商围绕技术标准与商业利益陷入了纷争，甚至到后来，纷争演变成了UNIX战争，为后来UNIX的衰弱埋下了伏笔。

从UNIX到自由软件

那么实际上在UNIX刚开始陷入纷争的时候，就已经有人开始思考了，既然UNIX的设计理念已经是成熟并且公开了的，注意我这里说的是设计理念而不是源代码，所以我们能不能从头开发一套类似的操作系统，不再依赖UNIX呢？

当然是能的，在20 世纪 80 年代，Richard Stallman 发起了 GNU 计划，希望开发一套与 UNIX 兼容，但允许用户自由使用、研究、修改和分享的操作系统。这就与那些计算机厂商恰好是相反的，因为计算机厂商修改后的UNIX系统肯定是不会把源代码给你的，因为他们要拿这个系统来赚钱，所以肯定不会给你源代码，但是这个计划，它的源代码是完全公开的，赋予了用户极大的自由。

虽然这个计划的名字是这三个字母G、N、U，但是好像不念做GNU，那天晚上我在宿舍里做PPT，我舍友看见了我PPT上的内容，问我为什么还讲genu计划，我当时听见这个名字不亚于听见了一声“牛来”，然后我才知道这个计划不念GNU，它念genu，但是无所谓了，我个人觉得怎么念都可以，等到大家大二的时候会学操作系统，很有可能还是会学RISC-V，这个怎么拼呢？就是R、I、S、C、-、V，但是它不念RISC-V，它念RISC-V，我们有一个操作系统老师，他第一次给我们上课时候，就说，不要念RISC-V，你要念RISC-V，你念RISC-V就跟你没学过这门课一样，再被人给笑话了。这是一个小插曲。

但是GNU的野心不止于此，GNU不仅希望能够开发出新的操作系统，更希望能够建立一种允许用户自由使用软件，研究源代码、分享改进成果的协作方式。换句话说，GNU试图构建一种用户占有软件自由与软件话语权的制度。下面我们来介绍GNU所设想的“自由软件”

自由软件并不是指不收费的软件，而是指能够保障用户使用、研究、修改和分享软件等基本自由的软件，换句话说就是让用户占有软件的绝大多数权益，让用户自行主导软件的未来发展，大家一起协作来促进软件的开发。

自由软件有四项最基本的自由：用户可以出于任何目的运行软件，用户可以研究软件如何工作，并根据自己的需要修改软件，用户可以复制和再分发软件，用户可以向他人分发修改后的版本。

自由软件不等于免费软件，实际上，如果你花100元购买了一款自由软件，你可以选择免费转发给其他人或者收费转发给其他人，这完全是你的自由。

那这时，自然而然地就引出了一个问题：自由软件的自由，究竟该怎么得到保障呢？

假设我们开发了一款自由软件，允许所有人使用、研究、修改和分享。但是，如果有人修改了我们的软件，随后把修改后的版本分发给别人，却不再向这些用户提供源代码，甚至不允许他们继续修改和分享，那后来的用户不就失去了原本应该享有的自由吗？

所以，为了让软件的自由能够一直延续下去，GNU 计划发展出了一种叫作 **Copyleft** 的许可机制，翻译过来叫作著佐权。

Copyleft 的核心思想其实很简单：我允许你使用、修改和分享软件，但是，如果你要把受 Copyleft 约束的软件或者它的修改版本分发给其他人，就必须继续保障接收者的相应自由。

那如果有人违反这些要求，会怎么样呢？

这里大家需要知道，Copyleft 并不只是开发者之间的一种道德约定，它实际上是借助著作权和具体的软件许可证来发挥作用的。

如果有人违反了许可证的要求，就可能失去许可证授予的相关权利，甚至承担著作权侵权责任。

不过，在实际处理违规问题时，自由软件基金会通常会优先联系违规者，帮助他们纠正问题。如果无法通过协商解决，才可能采取法律诉讼等手段。

那么，Copyleft 作为一种许可思想，究竟应该怎样落实到具体的软件中呢？

GNU 计划为此推出了 GNU GPL，也就是 GNU 通用公共许可证。

GPL 通过具体的法律条款，规定软件使用、修改和分发的条件，从而实现 Copyleft 希望达到的目标：**让自由不仅属于最初的用户，也能够延续到后来的用户。**

从GNU到Linux

前面我们介绍了 GNU 计划、自由软件以及 GPL。大家可能会觉得，GNU 计划好像仅仅提出了一套理念，怎么一直没看到GNU计划的实质性进展呢？其实是有进展的，在提出自由软件理念的同时，GNU 计划也在实实在在地开发软件。编写程序时使用的 GCC 编译器，在命令行中使用的 Bash，还有很多其他工具都被开发出来了，大家不认识这几个缩写也是没关系的，我们在后面的课程中会提到，到时会给大家很详细地讲解什么是GCC，什么是Bash。

但是，GNU 计划的最终目标是开发一套完整的自由操作系统，而仅仅拥有这些软件工具显然还不够。我们前面讲操作系统的时候提到过，操作系统需要管理处理器、内存以及各种硬件设备，而负责这些底层工作的核心部分，我们称为操作系统的内核，内核是一个操作系统最核心的部分。而当时呢，GNU计划虽然已经开发出了大量系统工具，但自己的内核始终没有达到成熟可用的状态。所以GNU计划所设想的操作系统迟迟发布不出来。

往往这种危急关头，总会有英雄出现对吧，电视剧里经常这么写，然后这个英雄就带着他的武器就来了，那这个人是谁呢？

Linux的诞生

这个人就是林纳斯托瓦兹，虽然我在PPT上写他对操作系统的工作原理产生了浓厚的兴趣，于是决定动手为自己的计算机开发一个操作系统内核，实际上并不是这样的，他在自传里面写的是，他好不容易攒钱买了一台自己的计算机，结果发现这个计算机上运行的操作系统是真的不好用，所以他决定自己写一个自己想用的出来，所以他在自己的房间里废寝忘食，写的昏天暗地，终于写出来了，如果我没记错的话就是这样，如果我记错了的话那我就记错了。总而言之就是这样。

顺带提一下林纳斯的这本自传叫做《只是为了好玩》，因为当时他做Linux也确实是为了好玩，而且他也没想到Linux后来能发展的如此壮大。

后来，在1991年的时候，林纳斯在互联网上介绍了自己的这个项目，并向其他开发者征求意见，他当时在介绍的时候，说这只是自己的一个业余项目，不会像GNU那样庞大和专业。

但是谁也没想到，这个项目其实在网络上引起了越来越多的人关注，大家都对这个项目表现出了极大的兴趣，积极地给这个内核提出各种修复意见和改进意见，Linux内核得到了极大的发展。

GNU与Linux相遇

我们刚才提到过，GNU 计划经过多年的开发，已经拥有了 GCC 编译器、Bash 以及各种基础命令等大量系统工具，但是 GNU 自己的操作系统内核还没有成熟。而 Linux 恰恰相反，它已经拥有了一个能够管理计算机硬件资源的内核，但是如果想让普通用户真正方便地使用计算机，仅仅依靠一个内核显然是不够的，还需要各种系统工具和其他软件。

所以，二者一拍即合，开发者将 Linux 内核与 GNU 提供的系统工具，以及其他必要的系统组件结合起来，最终形成了一套类 UNIX 操作系统，也就是我们这里所说的 GNU/Linux。

所以这里大家需要记住一点：**严格来说，Linux 指的是操作系统内核，而 GNU/Linux 指的是将 Linux 内核与 GNU 系统组件等软件结合起来形成的完整操作系统。** 不过在日常交流中，我们经常也会直接用 Linux 来称呼这类操作系统。

但是大家可能会有一个疑问，GNU计划不是倡导自由软件吗？那GNU和Linux结合在一起，那Linux是自由软件吗？

其实最开始的时候并不是。

Linux 最初虽然公开了自己的源代码，也允许其他开发者阅读、修改和分享，但是它最初使用的许可证有一个比较特殊的限制：不允许收费分发，甚至连复制和分发软件时收取的手续费都不允许。

因为我们前面提到过，自由软件中的“自由”并不等于免费。自由软件不仅允许用户免费分享，也允许用户在遵守许可证的前提下收费分发。因此，早期 Linux 的许可证实际上并不完全符合自由软件的定义。

不过，到了 1992 年，Linus Torvalds 做出了一项非常重要的决定：将 Linux 内核改为使用 GNU GPL 许可证。

这个 GPL，就是我们前面介绍过的、用于实现 Copyleft 机制的自由软件许可证。

改用 GPL 之后，Linux 的用户不仅能够研究、修改和分享源代码，还可以在遵守许可证要求的前提下进行商业分发。而且，当有人将受 GPL 约束的修改版本分发给其他人时，也需要按照规定继续保障接收者的相应自由。

这样一来，Linux 不仅在技术上能够与 GNU 的系统工具相互补充，在软件许可方面也与 GNU 所倡导的自由软件理念保持了一致。

随着 Linux 不断发展，越来越多的开发者开始关注它。有人在使用过程中发现了程序的错误，希望能够修复这些问题；有人觉得现有的功能还不够完善，希望增加一些新的功能；还有人希望能够把 Linux 移植到其他类型的计算机上，让更多的设备也能够运行 Linux。

但是这个时候就有问题了，这些开发者可能来自世界各地，彼此之间甚至从来没有见过面，他们又该如何共同维护同一个软件项目呢？

这就不得不提到我们的互联网了。

借助互联网，世界各地的开发者不再需要聚集在同一间办公室里，就可以通过讨论组(当然现在好像已经没有讨论组这种东西了)、电子邮件等方式交流自己的想法。如果有人发现了问题，或者希望增加某项功能，就可以先修改自己获得的源代码，再将这些修改整理成补丁，也就是 Patch，提交给项目的维护者。

维护者收到这些修改之后，并不是直接把所有人的代码都加入 Linux，而是需要经过讨论、代码审查以及测试等过程。只有被认为合适的修改，才有机会被合并到项目中。

而 Linux 的发展也逐渐向我们证明了一件事情：一个大型软件项目，并不一定只能由某一家企业内部的开发团队完成。只要具备开放的许可、便利的交流渠道以及有效的社区协作机制，世界各地的开发者同样可以共同推动一项技术的发展。

所以我们现在再回头看 Linux，它已经不再只是 Linus Torvalds 一个人的业余项目，而是逐渐成为了世界各地开发者共同参与的软件项目，这是一个十分伟大的进展。

Linux逐渐形成自己的生态

随着Linux发展的越来越好，使用范围越来越广，这时候就有人开始思考了，对于学计算机的人而言使用Linux是不困难的，但是对于有特定需求的普通用户，他们还需要自己安装各种系统工具、程序库和应用软件，那这样的话使用Linux的门槛依然是很高的。

所以，为了解决这个问题，一些开发者开始尝试把 Linux 内核、GNU 提供的系统工具，以及其他必要的软件直接组合起来，形成一套相对完整、方便安装和使用的操作系统。

这样的软件组合，就被称为 Linux 发行版，英文叫作 Linux Distribution。

从 20 世纪 90 年代开始，一批具有代表性的 Linux 发行版陆续出现。

我们下面列举了三个当时比较具有代表性的发行版。

这里大家可以注意一下，**虽然这些发行版都使用 Linux 内核，但它们并不完全相同。** 不同的开发团队可以根据自己的目标，选择不同的软件、配置方式以及维护策略，最终形成面向不同用户的操作系统。

而且，这里还有一个值得注意的地方：Linux 不仅吸引了个人开发者和自由软件社区的参与，也逐渐吸引了一些商业企业。就比如说这里的Red Hat Linux，就在逐渐探索商业化应用。

那么，经过这么多年的发展，Linux 发行版是不是就只有刚才提到的这几种呢?

当然不是。实际上，Linux 后来发展出了非常丰富的发行版生态，非常丰富的意思是根本数不清，实在是太多了。

这里我们就来详细介绍四个比较有代表性的 Linux 发行版。

首先是 Ubuntu。

Ubuntu 诞生于 2004 年，是一个基于 Debian 开发的 Linux 发行版。它最突出的特点之一，就是比较注重易用性，希望让更多普通用户能够方便地安装和使用 Linux。

大家可以想象一下，如果我们想学习 Linux，但是安装一个操作系统就需要输入大量命令，还需要自己配置各种软件，那么对于刚刚接触计算机的同学来说，门槛显然是比较高的。

而 Ubuntu 就希望改善这种情况。它提供了相对完善的图形化安装程序、桌面环境和软件管理工具，让用户不需要掌握太多底层知识，就能够完成日常操作。

另外，Ubuntu 还有一个非常重要的特点，就是它提供长期支持版本，也就是我们经常看到的 LTS 版本。Ubuntu 通常每两年发布一个 LTS 版本，为这些版本提供较长时间的维护和安全更新。因此，对于一些希望系统长期稳定运行的用户来说，Ubuntu 是一个比较合适的选择。

目前，Ubuntu 不仅被广泛应用于个人电脑，也经常出现在软件开发、服务器以及云计算等场景中。如果大家以后想尝试 Linux，Ubuntu 就是一个比较适合入门的选择。

接下来是 Debian。

实际上，我们上一页已经介绍过 Debian 了。它诞生于 1993 年，是一个非常有代表性的社区驱动的 Linux 发行版。

Debian 有两个比较突出的特点。

第一个特点是它非常重视自由软件的理念。Debian强调对用户自由的尊重，并希望通过社区协作来开发和维护操作系统。

第二个特点就是稳定性。

Debian 的软件开发分为不同的分支，其中比较有代表性的是 Stable，也就是稳定版。稳定版中的软件通常经过了比较充分的测试，因此软件版本可能不是最新的，但整个系统的行为通常更加稳定、可预期。

这里需要注意，稳定并不意味着 Debian 永远不会出现错误，而是说，它更加重视减少不必要的变化，以及保证不同软件之间的兼容性。

所以，Debian 经常被用于服务器等对系统稳定性要求比较高的环境。

而且，我们刚才提到过，Ubuntu就是基于Debian开发的，Debian 与 Ubuntu 有着非常密切的关系，它们都使用 APT 这样的软件包管理工具，有些同学现在可能还不知道什么是APT，完全没有关系，以后在课程实验中会碰到。

这也体现了开源软件的一个重要特点：后来的开发者可以在已有项目的基础上继续创新，而不必从零开始开发所有组件。

第三个是 Fedora。

Fedora 同样是一个由开源社区推动的 Linux 发行版，并且得到了 Red Hat，也就是红帽公司的重要支持。

大家还记得上一页介绍的 Red Hat Linux 吗？Fedora 与它有着密切的历史联系。

后来，Red Hat 逐渐形成了不同的发展方向：一方面是由社区推动、积极采用新技术的 Fedora；另一方面则是更加注重企业级需求、长期维护和商业支持的 Red Hat Enterprise Linux。

Fedora 最突出的特点，就是它比较积极地采用新的开源技术。

例如，当一些新的桌面技术、系统组件或者开发工具逐渐成熟时，Fedora 往往会比较早地将它们整合到自己的系统中，让用户能够体验到新的功能。

Fedora 通常大约每半年发布一个新版本，因此与 Debian Stable 相比，它的软件版本往往更新一些。当然，这也意味着用户需要更加频繁地进行系统升级。

最后一个是 Arch Linux。

Arch Linux 诞生于 2002 年。与刚才介绍的几个发行版相比，它的设计理念有些不同。

Arch Linux 比较强调简洁性和高度可定制，希望让用户能够按照自己的需要，自主选择和配置操作系统中的各种组件。

比如说，有些同学可能只希望拥有一个能够运行命令行程序的系统，不需要图形桌面；而另一些同学可能希望安装特定的桌面环境，或者自己选择各种系统工具。

Arch Linux 就比较强调把这些选择交给用户。

不过，这里的简洁并不意味着使用起来一定简单。恰恰相反，因为它通常需要用户自己完成更多配置，所以对于没有 Linux 基础的同学来说，学习成本可能比较高。

Arch Linux 还有一个非常有代表性的特点，叫作滚动更新。

什么意思呢?

我们刚刚介绍的 Ubuntu、Debian 和 Fedora，通常都会定期推出新的发行版本。用户使用一段时间后，可能需要把系统从一个发行版升级到下一个发行版。

但是 Arch Linux 不太一样。它没有这种固定周期的大版本发布方式，而是通过持续更新软件包，让用户不断获得新的软件版本。

这种方式的好处是，用户通常能够比较及时地使用较新的软件；但由于系统组件更新得比较频繁，用户也需要关注更新过程中可能出现的兼容性问题。

除此之外，Arch Linux 还有一个非常值得介绍的资源，叫作 ArchWiki。

它是由社区共同维护的技术文档网站，里面有大量关于 Linux 安装、配置、软件使用和问题排查的资料。即使我们没有使用 Arch Linux，也可以从中学习很多 Linux 相关的知识。

当然，Linux 发行版远远不止这四种。我们这里介绍它们，只是希望大家能够理解：**Linux 并不是只有一种固定的使用方式，而是可以根据不同用户的需求，发展出各种不同的操作系统。**

我们前面不是还提到过UNIX吗？UNIX厂商互相竞争的时候Linux逐渐发展起来了，就导致UNIX市场逐渐被Linux取代了。

那有的同学这个时候可能会问了，我们这不是开源课程吗？为什么这么讲了这么长时间了还是没有提到开源这两个字呢？我们现在把时间拨回到20世纪90年代

随着计算机技术的发展，个人计算机开始逐渐普及，越来越多的普通人拥有了编写软件的条件。与此同时，互联网的发展让世界各地的开发者能够方便地交流代码、分享成果，甚至共同开发同一个软件项目。

在这两方面的作用下，人们逐渐认识到，开放源代码不仅能够保障用户的自由，还能汇聚不同开发者的力量，共同改进软件。1998 年，一部分软件界人士开始推广“开源软件”这一名称，希望强调开放协作的实际价值，吸引更多开发者和企业参与。

于是，他们开始积极推广一个新的名称，叫作 **Open Source Software，也就是开源软件。**希望强调开放协作的实际价值，吸引更多开发者和企业参与。

那么，大家可能会有一个疑问：我们前面不是已经介绍过自由软件了吗？为什么现在又出现了一个开源软件？难道它们是两种完全不同的软件吗？

其实并不是。

自由软件和开源软件在具体的软件许可要求上有着非常高的重合度，很多我们今天熟悉的开源软件，同时也是自由软件。它们之间最主要的区别，并不在于软件本身，而在于各自所强调的理念。

自由软件运动更加关注用户应当拥有的基本自由。比如，我们前面介绍过的四项基本自由，就是希望用户能够自由地使用、研究、修改和分享软件。

而开源运动则更加注重开放协作所带来的实际价值。比如，通过公开源代码，让更多开发者能够共同发现问题、修复错误、增加功能，从而推动软件的发展。

所以，我们可以简单地理解为：**自由软件更加关注用户的权利，而开源软件更加关注开放协作的价值。**

当然，这并不是说自由软件不重视协作，或者开源软件不保障用户的自由，只是两者在宣传和阐述理念的时候，侧重点有所不同。

讲到这里，我们已经从最开始的操作系统，一路介绍了 UNIX、GNU、自由软件，以及 Linux 的诞生和发展，最后又讲到了开源运动的兴起。

那么大家可能还会有一个问题：我们讲了这么多历史，介绍了这么多理念，但是开源对于我们这些普通大学生来说，究竟意味着什么？

假设我们学校有一位学长，他开发了一款非常实用的课表管理软件，但毕业后没有时间继续维护。后来的同学发现了问题，还希望增加新的功能。难道这个软件就只能停止更新了吗？

如果这是一款按照合适的开源许可证发布的软件，情况可能就不一样了。

首先是**知识共享**。因为软件的源代码是公开的，后来的同学就可以阅读这些代码，学习这位学长当初是怎么设计和实现这些功能的。对于刚刚接触编程的同学来说，这其实也是一种非常有价值的学习方式。

其次是**协作创新**。如果有人在使用过程中发现了问题，就可以尝试修改代码、修复错误；如果有人产生了新的想法，也可以在原有项目的基础上增加新的功能。不同开发者还可以相互交流，把各自的改进成果分享出来，让软件变得越来越完善。

最后是**自主维护**。即使最初开发软件的学长已经毕业，不再维护这个项目，其他同学仍然有权在许可证允许的范围内继续开发它。这个项目的发展，就不必完全依赖最初的作者。

所以大家可以发现，开源不仅是公开代码，更是通过开放的许可与协作，让更多人有机会参与软件的学习、改进与长期发展。

当然，开源并不意味着一定会有人主动接手维护，也不意味着所有开源项目都能一直发展下去。但是，至少它为后来的开发者保留了继续参与和改进的可能性。

我最后想说些什么呢？

我们知道，并不是所有人都拥有同样的学习条件。有些人可能很早就接触了计算机，有条件购买设备、参加培训，也能够得到专业人士的指导。而另一些人，可能直到进入大学，才第一次有机会系统地学习编程和计算机知识。

所以我认为，开源的一个重要意义，就是在一定程度上降低这些门槛，它不能消除所有的不平等，但是它能够降低技术的获取门槛，打破部分知识壁垒，让更多的人有机会参与技术的发展。

这也是我们设计《这就是开源》这门课程的重要初衷。我们希望通过后面的课程，让大家逐渐掌握一些真正实用的计算机工具，也希望大家能够在未来尝试参与开源社区，甚至创造属于自己的开源项目。

谢谢大家！